RESUMEN EJECUTIVO · PROPUESTA PRÁCTICA

El Efecto Bullet

Deuda Cognitiva y la Ilusión de la Velocidad en la Era de la IA

Autor: Ariel Fabian · **Fecha:** Octubre de 2026 · **Versión:** Síntesis ejecutiva del trabajo de investigación

El problema

La IA genera código en segundos. Eso no quiere decir que el equipo lo entienda.

Antes, el ritmo lento del desarrollo manual ayudaba sin que nadie lo planificara. Para escribir un componente había que pensarlo, y mientras se pensaba se armaba un modelo mental del sistema. Naur (1985) lo llamó la *teoría del programa*: lo que el programador sabe sobre cómo el software se corresponde con el problema real. Sin esa teoría, el sistema no se puede modificar con seguridad.

La IA acelera la parte de escribir y saltea la de entender. El código aparece prolijo y parece progreso. Pero quien lo revisa no participó en las decisiones: no consideró alternativas ni descartó caminos. Cuando el ritmo sube, la revisión se reduce a mirar el código por arriba y aprobar. A lo que se acumula así lo llamamos **deuda cognitiva**: sistemas que funcionan, pero que nadie sabe modificar con seguridad.

Qué dicen los datos

Ninguno de estos estudios prueba la tesis por sí solo. Juntos sugieren un patrón.

- **He y Vasilescu (2026):** en 196.000 pull requests, la revisión humana profunda bajó del 48% al 21%.
- **Shen y Tamkin (2025):** desarrolladores que aprendían una librería nueva y delegaron el código en la IA sacaron 17 puntos porcentuales menos en comprensión (50% contra 67%).
- **Shaw y Nave (2026):** bajo presión de tiempo, las chances de aceptar respuestas erróneas de la IA se multiplicaron por más de tres.
- **Poulidis et al. (2025):** en un estudio con alumnos de ajedrez, los que podían pedir la solución a la IA cuando quisieran mejoraron 30%. Los que tenían la ayuda bloqueada hasta intentar solos mejoraron 64%.
- **Faros AI (2026), informe de industria:** en organizaciones con adopción masiva y sin reglas, los cambios aprobados sin revisión subieron 76,3% y los incidentes críticos en producción 125,4%.

Por qué el ajedrez

El ajedrez tiene décadas de investigación sobre memoria, decisiones con poco tiempo y trabajo con máquinas. Lo usamos como analogía, no como prueba. El ajedrez es un sistema cerrado. El software es abierto: los requerimientos son ambiguos y las normas cambian. Creemos que por eso los riesgos en software son mayores, pero es una conjetura que hay que probar.

Cinco reglas

Están adaptadas del ajedrez. Son propuestas, no resultados.

1. Control de ritmo

Un script descartable o una maqueta toleran un desarrollo rápido. Un motor de facturación, la lógica fiscal o un algoritmo transaccional con concurrencia, no.

- Clasificar los módulos antes de empezar: periféricos o críticos.
- En los críticos, el diseño se piensa a ritmo clásico. La IA puede explorar alternativas.
- En los críticos no se mide al equipo por tickets cerrados, pull requests aprobados o líneas escritas. Se mide por estabilidad en producción y por cuánto entiende el equipo.

2. Regla del 10 + 10

La IA genera en 4 horas lo que antes llevaba 40, y se usan las 36 horas "ahorradas" en la próxima tarea. Nadie lee lo que se generó.

La regla: por cada hora de generación en un componente crítico, se reserva otra para que el desarrollador lea, entienda y verifique.

> Ejemplo: una funcionalidad estimada en 40 horas manuales se genera en 4 horas (10%) y se reservan otras 4 para entenderla (10%). Se entrega en 8 horas, el 20% del esfuerzo original: cinco veces más rápido.

El factor 10× es un supuesto de ejemplo. En sistemas grandes y maduros la ganancia es menor, y la reserva se ajusta.

Hay un respaldo en la literatura de inspección. Cohen et al. (2006) vieron que, por encima de unas 450 líneas por hora, la mayoría de las revisiones detectaba menos defectos que el promedio (es una correlación: también podría ser código más simple). Fagan (1976) documentó ritmos de 100 a 150 líneas por hora para lógica compleja. Usamos 100 a 200 líneas de lógica densa por hora como referencia.

3. Peaje cognitivo

En los componentes críticos, antes del commit se pausa la generación de código. Un agente interroga al desarrollador sobre el cambio, y este responde solo, sin ayuda del modelo. Tres preguntas, derivadas de Naur:

1. **Datos e invariantes.** ¿Cómo fluye la información entre los componentes que cambiaron? ¿Qué invariantes del negocio garantiza este cambio?
2. **Supuestos de falla.** ¿Qué supuso la solución sobre la red, los accesos simultáneos y los valores nulos? ¿Qué pasa si uno falla?
3. **Cambio de negocio.** Si mañana cambia la regla central, ¿qué partes del código hay que tocar y por qué?

La herramienta es una skill para el IDE o la terminal:

1. Mira el diff sin resumirlo, para no inducir la sensación de entender.
2. Calcula el tiempo mínimo de revisión (100 a 200 líneas de lógica por hora y 10 a 12 minutos por punto de decisión; esto último es una estimación propia, a calibrar).
3. Hace las tres preguntas sobre archivos y métodos concretos del diff.
4. Evalúa con un semáforo. Verde: habilita el commit. Amarillo: señala el punto ciego y pide otra respuesta. Rojo: bloquea el commit y guía un recorrido por los bloques críticos.

4. Auditoría preventiva

La IA tiende a resolver el camino feliz: datos válidos, red sin latencia, sin accesos simultáneos. Las pruebas unitarias básicas suelen probar ese mismo camino. En producción aparecen datos sucios, cortes y concurrencia. Antes de desplegar un cambio crítico, se audita:

- **Concurrencia e idempotencia:** transacciones atómicas, reintentos que no dupliquen cobros ni registros, sin deadlocks bajo carga real.
- **Datos reales:** campos nulos, registros históricos incompletos, caracteres especiales. Si algo falla, que falle de forma clara, nunca en silencio.
- **Aislamiento de la lógica de negocio:** las reglas separadas de la infraestructura, en funciones que se puedan verificar solas.

5. Modelo centauro

En el torneo Freestyle de 2005, dos aficionados con computadoras comunes y un proceso de trabajo riguroso vencieron a grandes maestros y a supercomputadoras manejadas sin método. Kaspárov observó que el proceso puede pesar más que la potencia de la herramienta. Hoy los motores juegan mejor solos en ajedrez, pero en software el dominio es abierto y la IA no garantiza una visión estratégica integral.

La regla: el equipo conserva la arquitectura y las decisiones de diseño. El uso de la IA se rige por reglas compartidas. El programador conduce y no compite en velocidad con la IA.

Dos objeciones

"Con pull requests chicos alcanza." Achicar el lote ayuda y Cohen et al. lo recomiendan. Pero con IA el problema pasa a ser el ritmo: veinte PRs chicos por día pueden generar la misma fatiga y la misma aprobación pasiva. La regla del 10 + 10 no compite

con los PRs chicos: los complementa fijando un tope al ritmo.

"Un desarrollador responsable se toma el tiempo solo." Puede intentarlo. Pero si las métricas de seguimiento se mantienen atadas al volumen de tickets cerrados, quien frena a estudiar el código puede parecer menos veloz que quien avanza sin revisar en profundidad. Por eso la reserva funciona mejor como una pauta explícita del equipo y no como un esfuerzo aislado.

Límites

- **Falsos verdes.** Si el examen lo hace un agente de IA, un desarrollador puede dar respuestas verosímiles y superficiales. Se reduce anclando las preguntas a líneas concretas del diff y con revisiones cruzadas entre pares.
- **Fricción.** Un peaje completo para scripts o cambios cosméticos sobra y se va a esquivar. Va solo en lo crítico.
- **Alineación.** Si los indicadores solo miran el volumen inmediato, el peaje corre el riesgo de volverse un trámite.
- **Parámetros.** El 10 + 10 y los 10 a 12 minutos son propuestas iniciales, sin validación independiente.

Cómo probarlo

Un diseño factorial 2 × 2 con equipos reales, seguimiento de al menos seis meses y 4 a 6 equipos por brazo. Es exploratorio: sin cálculo de potencia formal, es probable que solo detecte efectos grandes.

Base común para los cuatro brazos: clasificación de módulos, auditoría preventiva, PRs chicos, tests automáticos y revisión por pares.

Brazo	Reserva de tiempo (10 + 10)	Peaje cognitivo
A (control)	No	No
B	Sí	No
C	No	Sí
D	Sí	Sí

Se mide: incidentes críticos en producción, tiempo medio de recuperación (indicador indirecto de comprensión), retrabajo en las dos semanas posteriores, lead time y throughput (para ver si el tiempo extra se compensa), y una evaluación ciega de comprensión a los 30 días.

Qué falta

Evidencia propia. Todo lo anterior es una hipótesis de trabajo. Si querés probar las reglas en tu equipo, aunque sea una por vez en un módulo crítico, ese dato vale más que cualquier argumento.

Fuentes citadas

- Cohen, J., Brown, E., DuRette, B., & Teleki, S. (2006). *Best Kept Secrets of Peer Code Review.* SmartBear Software & Cisco Systems.
- Fagan, M. E. (1976). Design and code inspections to reduce errors in program development. *IBM Systems Journal, 15*(3), 182-211.
- Faros AI. (2026). *The Speed Trap: How Generative AI Impacts Software Quality and Production Incidents.* Faros Engineering Report.
- He, H., & Vasilescu, B. (2026). *AI Writes Faster Than Humans Can Review: A Longitudinal Study of an Enterprise "2x" Mandate.* Carnegie Mellon University & Stanford University (arXiv:2607.01904).
- Naur, P. (1985). Programming as Theory Building. *Microprocessing and Microprogramming, 15*(5), 253-261.
- Poulidis, S., Bastani, H., & Bastani, O. (2025). *Self-Regulated AI Use Hinders Long-Term Learning.* The Wharton School Research Paper (SSRN 5604932).
- Shaw, S. D., & Nave, G. (2026). *Thinking—Fast, Slow, and Artificial: How AI is Reshaping Human Reasoning and the Rise of Cognitive Surrender.* Wharton School, University of Pennsylvania (SSRN 6097646).
- Shen, J. H., & Tamkin, A. (2025). *How AI Impacts Skill Formation.* Anthropic Research Report (arXiv:2601.20245).